FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Michael Dominic L. Liggayu
Section:	An42
Professor:	Ms. Crisola Tan
Date:	Oct 5, 2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

```
Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella
```

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
   RBR was easier to implement than CBR because it only uses IF-Then rules. We only need to check the student's condition and provide an appropriate study recommendation.
2. Which is more flexible in solving unstructured problems?
   CBR is more flexible since we can find a similar past case and reuse the solution with the new problem.
3. Which is better for situations with continuously changing data?
   CBR is better because new problems and solution can be added to the case base allowing the system to use more previous experience when solving future problems.
4. Examples of real-world applications for both.
   RBR can be used in troubleshooting systems, expert systems, and technical support. CBR can be used in recommendation systems, customer support, and medical case analysis.

Your Code - Laboratory Tasks ⇩

Source code:

```
# CS0065 - Intelligent Systems
# Technical Assessment 4
#Study recommendation system

# PART 1 - KNOWLEDGE REPRESENTATION

print("=========================================")
print("       STUDY RECOMMENDATION SYSTEM")
print("=========================================")

# Get the student's current study conditions
study_time = input("Enter study time (short/long): ").lower()
energy = input("Enter energy level (low/normal): ").lower()
difficulty   =   input("Enter   difficulty   (easy/normal/hard):
").lower()
focus = input("Enter focus level (low/normal/high): ").lower()
exam_near = input("Is the exam near? (yes/no): ").lower()

# Convert exam answer to True or False
if exam_near == "yes":
    exam_near = True
else:
    exam_near = False

# Knowledge Base
# Stores facts about the student's current study condition
facts = {
    "study_time": "short",
    "energy": "low",
    "difficulty": "hard",
    "focus": "low",
    "exam_near": True
}

print("KNOWLEDGE BASE")
print(facts)

# PART 2 - RULE-BASED REASONING

# Rule 1
def rule_short_time(facts):
    if facts["study_time"] == "short":
```

```
        return "Focus on the most important topics."

# Rule 2
def rule_low_energy(facts):
    if facts["energy"] == "low":
        return "Take a short break before studying."

# Rule 3
def rule_hard_subject(facts):
    if facts["difficulty"] == "hard":
        return "Study the difficult topics first."

# Rule 4
def rule_low_focus(facts):
    if facts["focus"] == "low":
        return "Study in a quiet place and remove distractions."

# Rule 5
def rule_exam_near(facts):
    if facts["exam_near"] == True:
        return "Review your notes and practice questions."

# Inference Engine
rules = [
    rule_short_time,
    rule_low_energy,
    rule_hard_subject,
    rule_low_focus,
    rule_exam_near
]

print("\nRULE-BASED REASONING")
print("Study Recommendations:")

for rule in rules:
    result = rule(facts)

    if result:
        print("-", result)


# PART 3 - CASE-BASED REASONING

# Past Cases
case_base = [
```

```
    {
        "problem": {
            "study_time": "short",
            "energy": "normal",
            "difficulty": "hard"
        },
         "solution": "Focus on the most important and difficult
topics."
    },

    {
        "problem": {
            "study_time": "long",
            "energy": "low",
            "difficulty": "normal"
        },
           "solution": "Take breaks and study using shorter
sessions."
    },

    {
        "problem": {
            "study_time": "short",
            "energy": "low",
            "difficulty": "hard"
        },
          "solution": "Take a short break, then focus on the
difficult topics."
    }
]

# New Problem
new_problem = {
    "study_time": "short",
    "energy": "low",
    "difficulty": "hard"
}

# RETRIEVE

best_case = None
highest_score = 0

for case in case_base:
```

```
    score = 0

    for feature in new_problem:

        if new_problem[feature] == case["problem"][feature]:
            score += 1

    if score > highest_score:
        highest_score = score
        best_case = case


# REUSE

solution = best_case["solution"]


# REVISE


revised_solution = solution


# RETAIN

new_case = {
    "problem": new_problem,
    "solution": revised_solution
}

case_base.append(new_case)


# DISPLAY CBR RESULT

print("\nCASE-BASED REASONING")

print("New Problem:", new_problem)

print("Most Similar Case:", best_case["problem"])

print("Similarity Score:", highest_score)

print("Reused Solution:", solution)
```

```
print("Revised Solution:", revised_solution)

print("New case added to the case base.")
```

Output:

```
========================================
       STUDY RECOMMENDATION SYSTEM
========================================
Enter study time (short/long): short
Enter energy level (low/normal): low
Enter difficulty (easy/normal/hard): normal
Enter focus level (low/normal/high): normal
Is the exam near? (yes/no): yes
KNOWLEDGE BASE
{'study_time': 'short', 'energy': 'low', 'difficulty': 'hard', 'focus': 'low', 'exam_near': True}

RULE-BASED REASONING
Study Recommendations:
- Focus on the most important topics.
- Take a short break before studying.
- Study the difficult topics first.
- Study in a quiet place and remove distractions.
- Review your notes and practice questions.

CASE-BASED REASONING
New Problem: {'study_time': 'short', 'energy': 'low', 'difficulty': 'hard'}
Most Similar Case: {'study_time': 'short', 'energy': 'low', 'difficulty': 'hard'}
Similarity Score: 3
Reused Solution: Take a short break, then focus on the difficult topics.
Revised Solution: Take a short break, then focus on the difficult topics.
New case added to the case base.
PS D:\Feu Alabangseu SY 26 - 27 4th year\first sem\CS Elective - Intelligent Systems\Lab act\Technical Assessment 4>
```

```
PS D:\Feu Alabangseu SY 26 - 27 4th year\first sem\CS Elective - Intelligent Systems\Lab act\Technical Assessment 4> python -u "d:\
e - Intelligent Systems\Lab act\Technical Assessment 4\tempCodeRunnerFile.py"
========================================
       STUDY RECOMMENDATION SYSTEM
========================================
Enter study time (short/long): long
Enter energy level (low/normal): low
Enter difficulty (easy/normal/hard): easy
Enter focus level (low/normal/high): high
Is the exam near? (yes/no): no
KNOWLEDGE BASE
{'study_time': 'short', 'energy': 'low', 'difficulty': 'hard', 'focus': 'low', 'exam_near': True}

RULE-BASED REASONING
Study Recommendations:
- Focus on the most important topics.
- Take a short break before studying.
- Study the difficult topics first.
- Study in a quiet place and remove distractions.
- Review your notes and practice questions.

CASE-BASED REASONING
New Problem: {'study_time': 'short', 'energy': 'low', 'difficulty': 'hard'}
Most Similar Case: {'study_time': 'short', 'energy': 'low', 'difficulty': 'hard'}
Similarity Score: 3
Reused Solution: Take a short break, then focus on the difficult topics.
Revised Solution: Take a short break, then focus on the difficult topics.
New case added to the case base.
PS D:\Feu Alabangseu SY 26 - 27 4th year\first sem\CS Elective - Intelligent Systems\Lab act\Technical Assessment 4>
```

```
Terminal

PS D:\Feu Alabangseu SY 26 - 27 4th year\first sem\CS Elective - Intelligent Systems\Lab act\Technical Assessment 4> python
e - Intelligent Systems\Lab act\Technical Assessment 4\TA4_Liggayu.py"
========================================
       STUDY RECOMMENDATION SYSTEM
========================================
Enter study time (short/long): short
Enter energy level (low/normal): normal
Enter difficulty (easy/normal/hard): normal
Enter focus level (low/normal/high): high
Is the exam near? (yes/no): yes
KNOWLEDGE BASE
{'study_time': 'short', 'energy': 'low', 'difficulty': 'hard', 'focus': 'low', 'exam_near': True}

RULE-BASED REASONING
Study Recommendations:
- Focus on the most important topics.
- Take a short break before studying.
- Study the difficult topics first.
- Study in a quiet place and remove distractions.
- Review your notes and practice questions.

CASE-BASED REASONING
New Problem: {'study_time': 'short', 'energy': 'low', 'difficulty': 'hard'}
Most Similar Case: {'study_time': 'short', 'energy': 'low', 'difficulty': 'hard'}
Similarity Score: 3
Reused Solution: Take a short break, then focus on the difficult topics.
Revised Solution: Take a short break, then focus on the difficult topics.
New case added to the case base.
PS D:\Feu Alabangseu SY 26 - 27 4th year\first sem\CS Elective - Intelligent Systems\Lab act\Technical Assessment 4>
```